申请反欺诈评分查询服务（fqz）· API 接口文档与使用手册

> 面向接入方（客户）技术与运维人员的对外接口说明。
> 版本：fqz（申请反欺诈评分）｜通信：HTTP + JSON｜编码：UTF-8
> 说明：本服务采用统一的请求信封与 MD5 加签方式，响应分为 head（网关头部）与 body（业务结果）两部分。其余内部处理细节调用方无需关心。
> 关键特性：入参仅需手机号，可传明文，也可传手机号的 MD5 或 SHA256 摘要（由 encryptionType 指定，见 3.1.1）。申请反欺诈评分的查询结果是一个结构化对象，本服务将其原样序列化为 JSON 字符串置于 body.result.range，由调用方自行解析（见 3.1.5 result.range 结构说明）。
> 计费提示：本服务按有效查询次数计费——查得数据（001）与查无结果（999）均计费，详见 五、计费说明。

一、接入必读

1.1 适用范围

本文档适用于接入本平台「申请反欺诈评分查询服务（fqz）」的第三方产品技术开发人员与日常运维人员。

1.2 接入须知

1. 正式访问域名在接入时由我方商务提供。
2. 接入前需先申请开通账户，由我方分配 appKey 与 appSecret（加签密钥）。
3. 接入时需向我方提供贵司的外网出口 IP，以便加入访问白名单。

1.3 接口说明

项目	说明
请求方式	POST（计费查询数查询为 GET）
通信协议	HTTP
数据格式	请求体与响应体均为 JSON
字符编码	UTF-8
超时时间	4 秒（建议客户端读超时 ⩾ 5 秒）
HTTP 状态码	恒为 200；业务结果与错误均通过响应体的 head.errorCode / body.code 表达
签名	调用时需对 body 中的业务参数 + 我方分配的 appSecret 进行 MD5 加签，详见 二、鉴权与加签

1.4 环境说明

正式环境：使用正式账户，调用已开通接口，按有效查询次数计费（见 五、计费说明）。

测试环境：使用测试账户，返回挡板/联调数据。

正式账户仅适用于正式环境，测试账户仅适用于测试环境。

二、鉴权与加签

所有业务接口共用同一套请求信封与鉴权方式。

2.1 请求信封（顶层参数）

参数	示例	类型	必填	说明
appKey	y8909fqz	String	是	我方分配给客户的公开标识
sign	5728ac48301ae03642bfd06c63b66edc	String	是	对 body 业务参数的 MD5 签名（见 2.3）
encryptionType	1	int	否	手机号传入形态：1 = 明文（默认）；2 = MD5 摘要；3 = SHA256 摘要。详见 3.1.1
body	{...}	Object	是	业务请求体，见各接口定义

注意：本服务不需要 apiKey 参数（产品由请求路径区分）。appKey / sign / encryptionType 不参与签名计算。encryptionType 缺省或为 1 时，body.mobile 传明文手机号；为 2 或 3 时，body.mobile 传对应的摘要值。

2.2 鉴权校验顺序

网关按以下顺序校验，任一失败立即返回对应 head.errorCode（直接拒绝，不计费）：

1. appKey 是否存在 → 否则 505001
2. appKey 是否匹配到账户 → 否则 505004
3. 账户是否有效（启用且在有效期内）→ 否则 505007
4. 签名是否正确 → 否则 505002

2.3 加签方式

sign 是对 body 业务参数计算的 MD5 签名，按以下四步生成：

1. 取出 body 中所有非空的业务参数（不含文件/字节流类型，不含值为空的参数）。
2. 按参数名（key）的 ASCII 升序排序；首字符相同则依次比较后续字符。
3. 将排序后的参数按「参数名参数值」直接拼接（键与值之间、各对之间均无分隔符），最后追加 appSecret，得到待签名串。
4. 对待签名串做 MD5，取 32 位小写十六进制字符串，赋值给 sign。

appKey / sign / encryptionType 属信封字段，不参与签名计算；参与计算的只有 body 内的业务参数，且取的是**实际放进 body 的值**——传摘要时就对摘要字符串加签。

完整示例（可直接复算验证）

以本接口唯一入参 mobile 为例。下方用一个示例密钥演示整个计算链路——正式 appSecret 由我方分配，替换为你自己的密钥后按同样步骤计算即可。各字段取值与最终 sign 严格对应，可用任意语言复算核对：

```
输入 body      : { "mobile": "17325421010" }
示例 appSecret : demoSecret2026          （示例值，正式密钥由我方分配）
① 取非空业务参数：mobile=17325421010
② 按 key 的 ASCII 升序排序：[mobile]（仅一个字段）
③ 拼接「参数名参数值」并追加 appSecret：
   待签名串 = mobile17325421010demoSecret2026
④ MD5(待签名串) 取小写 hex：
   sign = 5728ac48301ae03642bfd06c63b66edc
```

用任意语言对上面的「待签名串」做一次 MD5（UTF-8 编码、结果取 32 位小写 hex），都应得到完全相同的 sign 值。若结果不一致，请逐项核对：是否漏剔空值、是否按 ASCII 升序、是否 UTF-8 编码、hex 是否为小写。

操作方式补充

- 本接口 body 仅 mobile 一个字段，排序后待签名串即「`mobile` + body.mobile 的值 + `appSecret`」三段直接相连，中间无任何符号。
- 以摘要形式传手机号时（encryptionType = 2 / 3），body.mobile 的值就是摘要字符串，签名对摘要字符串计算，示例见 3.1.2。
- 若 body 为空对象 `{}`（如 3.2 计费查询数查询接口），没有业务参数参与拼接，待签名串退化为 appSecret 本身，即 `sign = MD5(appSecret)`。以示例密钥演示：`sign = MD5("demoSecret2026") = 7d31d1028b99e31b1e5b6049af4bd73c`。
- 请勿写死字段顺序：拼接顺序完全由 key 的 ASCII 升序决定，将来 body 若新增字段，顺序会自动变化，按算法重新排序即可，无需改动代码。

2.4 加签代码示例

Java

```java
public static String sign(Map<String, String> bodyParams, String appSecret) throws Exceptio
    StringBuilder sb = new StringBuilder();
    List<String> keys = new ArrayList<>(bodyParams.keySet());
    Collections.sort(keys); // ASCII 升序
    for (String k : keys) {
        String v = bodyParams.get(k);
        if (v == null || v.isEmpty()) continue; // 剔除空值
        sb.append(k).append(v);
    }
    sb.append(appSecret);
    MessageDigest md = MessageDigest.getInstance("MD5");
    byte[] digest = md.digest(sb.toString().getBytes(StandardCharsets.UTF_8));
    StringBuilder hex = new StringBuilder();
    for (byte b : digest) hex.append(String.format("%02x", b));
    return hex.toString();
}
```

Python

```python
import hashlib
def sign(body_params: dict, app_secret: str) -> str:
    parts = []
    for k in sorted(body_params.keys()):  # ASCII 升序
        v = body_params[k]
        if v is None or v == "":
            continue                      # 剔除空值
        parts.append(f"{k}{v}")
    raw = "".join(parts) + app_secret
    return hashlib.md5(raw.encode("utf-8")).hexdigest()  # 小写 hex
```

Go

```go
func sign(body map[string]string, appSecret string) string {
    keys := make([]string, 0, len(body))
    for k, v := range body {
        if v != "" { // 剔除空值
            keys = append(keys, k)
        }
    }
    sort.Strings(keys) // ASCII 升序
    var sb strings.Builder
    for _, k := range keys {
        sb.WriteString(k)
        sb.WriteString(body[k])
    }
    sb.WriteString(appSecret)
    sum := md5.Sum([]byte(sb.String()))
    return hex.EncodeToString(sum[:]) // 小写 hex
}
```

三、接口列表

3.1 申请反欺诈评分查询（fqz）

项目	内容
路径	POST /v1/openapi/zlx/querySrmxFQZ
完整地址	http://www.aiszcloud.cn:8080/v1/openapi/zlx/querySrmxFQZ
鉴权	appKey + MD5 签名（见第二章）

3.1.1 请求 body 参数

参数	示例	类型	必填	说明
mobile	17325421010	String	是	国内手机号，按信封 encryptionType 传明文或摘要，见下表

mobile 的三种传入形态（三选一，由信封 encryptionType 指定）：

encryptionType	body.mobile 传什么	格式要求	示例（手机号 17325421010）
1 或不传	明文手机号	1 开头 11 位纯数字	17325421010
2	手机号的 MD5 摘要	32 位十六进制	043b5e7b9e2f9af9ddc61c3f4c5f51c5
3	手机号的 SHA256 摘要	64 位十六进制	b3c902cf670a95f4f99c5e103bf177cce1b1970732b89007aa5c975004b4871e

摘要的计算方式：对 11 位明文手机号（不含空格、+86 等前缀）按 UTF-8 编码直接计算 MD5 / SHA256，**不加盐**，取十六进制字符串。推荐小写，大写同样接受。

```
import hashlib
mobile = "17325421010"
md5_mobile    = hashlib.md5(mobile.encode("utf-8")).hexdigest()      # encryptionType = 2
sha256_mobile = hashlib.sha256(mobile.encode("utf-8")).hexdigest()   # encryptionType = 3
```

```
// Java：MessageDigest.getInstance("MD5") 或 ("SHA-256")，对 mobile.getBytes(UTF_8) 计算后转小
```

本接口仅需 mobile 一项，不需要姓名与身份证号；额外传入的字段不参与校验、亦不影响结果。

以下情况将返回 head.errorCode = 505062，直接拒绝，不计费：mobile 缺失；encryptionType 为 1 时不是 1 开头的 11 位数字；为 2 时不是 32 位十六进制；为 3 时不是 64 位十六进制；encryptionType 不是 1 / 2 / 3。

注意：摘要不可逆，本服务只能校验摘要的格式，无法核对它是否由正确的手机号算出。摘要算错（如多了空格、加了前缀、用错算法）时请求仍会正常执行，结果为查无（999）并**照常计费**。上线前请先用上表示例自检摘要计算。

3.1.2 请求示例

① 明文手机号（encryptionType = 1）

```
{
  "encryptionType": 1,
  "appKey": "y8909fqz",
  "sign": "5728ac48301ae03642bfd06c63b66edc",
  "body": {
    "mobile": "17325421010"
  }
}
```

② MD5 摘要（encryptionType = 2）

```
{
  "encryptionType": 2,
  "appKey": "y8909fqz",
  "sign": "cce3c706f050f17546ced9f731524f32",
  "body": {
    "mobile": "043b5e7b9e2f9af9ddc61c3f4c5f51c5"
  }
}
```

此时待签名串 = `mobile043b5e7b9e2f9af9ddc61c3f4c5f51c5demoSecret2026`。

③ SHA256 摘要（encryptionType = 3）

```
{
  "encryptionType": 3,
  "appKey": "y8909fqz",
  "sign": "36c79776fea0cd7c8dbcf2a8c19ab365",
  "body": {
    "mobile": "b3c902cf670a95f4f99c5e103bf177cce1b1970732b89007aa5c975004b4871e"
  }
}
```

此时待签名串 = `mobileb3c902cf670a95f4f99c5e103bf177cce1b1970732b89007aa5c975004b4871edemoSecret2026`。

说明：以上 sign 均由 2.3「完整示例」的示例密钥 demoSecret2026 计算得到；接入时请替换为我方分配的正式 appSecret 并按同样算法重新计算。三种形态的响应结构完全相同。

3.1.3 响应结构

响应分为 head（网关头部）与 body（业务结果）两部分：

head 字段：

参数	示例	类型	说明
errorCode	0	String	网关返回码。0 = 成功（含查得/查无）；非 0 = 网关级错误，此时无 body
errorMsg	success	String	返回描述
logId	a1b2c3...	String	全链路追踪 ID，排障/对账时请提供
time	128	Number	服务处理耗时（毫秒）
timestamp	1718456789012	Number	响应时间戳（毫秒）

body 字段（仅 head.errorCode = 0 时返回）：

参数	示例	类型	说明
code	001	String	业务结果码。001 = 查得数据【计费】；999 = 查无结果【计费】
msg	成功	String	业务描述
reqid	lkf9x2...	String	本次请求流水号
uid	01A15337-57C4-...	String	交易流水号（对账用）
result	{...}	Object	业务内容，仅 code = 001 时存在
result.range	"{\"score\":\"258\"}"	String	申请反欺诈评分结果的 JSON 字符串，调用方需 JSON.parse 后使用，结构见 3.1.5

3.1.4 响应示例

① 查得数据（计费）

```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d4", "time": 128, "time
  "body": {
    "code": "001",
    "msg": "成功",
    "reqid": "lkf9x2ab",
    "uid": "01A15337-57C4-5502-9FD2-7EFE64105F6B",
    "result": {
      "range": "{\"score\":\"258\"}"
    }
  }
}
```

② 查无结果（计费）

```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d5", "time": 96, "times
  "body": {
    "code": "999",
    "msg": "未查得",
    "reqid": "lkf9x2ac",
    "uid": "7C2E0A41-3B8D-5F17-A6C4-1D9B0E2F3A55"
  }
}
```

999 表示本次查询已完成、但无法对该手机号给出风险判断（未查得）——这是一个有效的查询结论，与 001 同样计费（见五、计费说明）。

③ 网关级错误（无 body）

```
{
  "head": { "errorCode": "505002", "errorMsg": "账号信息异常", "logId": "a1b2c3d6", "time": 5,
}
```

3.1.5 result.range 结构说明

result.range 是申请反欺诈评分结果对象序列化后的 JSON 字符串（完整结果原样返回，未做裁剪）。调用方应先对该字符串做一次 JSON.parse / 反序列化，再读取以下字段：

字段	类型	取值范围	说明
score	String	200 ~ 800	申请反欺诈评分，以字符串形式返回（如 "258"），使用前请转为整数。分值越大，风险越高

说明：result.range 的具体字段可能随数据能力升级扩展，本服务保证完整透出。建议调用方按「存在即解析、缺失即忽略」的方式做容错处理，避免因后续新增字段导致解析失败。评分如需按特定分值段使用，以双方商务沟通结果为准。

解析示例（JavaScript）：

```
if (resp.head.errorCode === "0" && resp.body.code === "001") {
  const range = JSON.parse(resp.body.result.range);
  const score = parseInt(range.score, 10); // 200~800，越大风险越高
  console.log(score);
}
```

3.2 计费查询数查询（扩展接口）

查询本账户累计计费查询的次数，用于客户侧自助监控与对账。不返回额度上限/剩余量（本服务无额度限制）。

项目	内容
路径	GET /v1/openapi/zlx/quotaFQZ
鉴权	与主接口一致（请求体中携带 appKey + sign 信封；body 可为 {}，此时待签名串退化为 appSecret 本身，即 sign = MD5(appSecret)，算法与示例见 2.3）

响应示例：

```
{
  "errorCode": "0",
  "errorMsg": "success",
  "status": "ACTIVE",
  "serviceUsed": 1280,
  "totalCalls": 1280
}
```

参数	说明
status	账户状态（ACTIVE/SUSPENDED 等）
serviceUsed	累计计费查询次数（code = 001 与 code = 999 均计入）
totalCalls	累计有效查询次数（含查得与查无；不含被网关拦截的请求）

说明：无任何额度上限拦截，仅做计费次数统计。本服务对查得与查无同等计费，故 serviceUsed 与 totalCalls 在正常情况下一致；对账请以 serviceUsed 为准。

3.3 健康检查

项目	内容
路径	GET /healthz
鉴权	无
响应	HTTP 200，纯文本 ok

四、返回码说明

4.1 网关返回码 head.errorCode

errorCode	含义	典型原因
0	成功	调用成功（业务结果见 body.code）
505001	appKey 异常	缺少或非法 appKey
505004	账户信息不存在	appKey 未匹配到账户
505007	服务尚未开通	账户停用 / 过期 / 未开通
505002	账号信息异常	签名校验失败
505003	产品编号异常	保留
505062	数据请求异常	参数非法 / 服务处理异常 / 超时未决 / 系统错误（默认错误码）

服务内部处理异常均统一返回 505062，不计费，并经异步复查兜底。

4.2 业务结果码 body.code（仅 errorCode = 0 时）

code	含义	是否计费
001	查得数据	计费
999	查无结果	计费

注：本服务按有效查询次数计费。只要本次查询产出了确定结论——无论是查得评分（001）还是无法判断风险（999）——均计费一次。仅当返回 head.errorCode 非 0（未能产出结论）时不计费。

五、计费说明

body.code = 001（查得数据）与 body.code = 999（查无结果）均计入计费查询次数并对客户计费。本服务按有效查询次数计费：查无结果同样消耗一次查询并产出确定结论。网关级错误（head.errorCode 非 0：鉴权失败、参数非法、系统异常等）一律不计费。计费以最终落库的台账为准，超时未决请求会经异步复查/对账裁定状态，不会重复计费。计费次数可随时通过计费查询数查询（3.2）接口自助核对。

六、使用手册（接入与最佳实践）

6.1 接入流程

1. 向商务申请账户，获取 appKey、appSecret、正式/测试域名，并提供贵司外网出口 IP 用于加白。
2. 确定手机号传入形态（明文 / MD5 / SHA256），按第二章实现加签，先在测试环境联调，再切正式环境。
3. 上线后通过计费查询数查询接口（3.2）监控调用量与计费次数。

6.2 幂等与重试

客户端建议为每笔查询设置合理超时（≥ 5 秒）。

收到网络超时/无响应时可安全重试：网关基于内部流水号做幂等，不会因重试重复计费。

请勿对已明确返回 head.errorCode 的请求做无差别重试（如参数错误 505062、鉴权错误 505001/505002/505004），应先修正再发起。

6.3 错误处理建议

现象	排查方向
505001 / 505004	检查 appKey 是否正确、是否用错环境账户
505002	检查签名算法（排序/空值剔除/UTF-8/小写 hex），可用 2.3「完整示例」复算自检；传摘要时确认是对摘要字符串加签
505007	联系商务确认账户状态与有效期
505062	检查 encryptionType 与 mobile 形态是否匹配（1 = 11 位手机号、2 = 32 位 hex、3 = 64 位 hex）；若入参正常仍持续出现，记录 logId 联系我方
摘要传入后大量 999	检查摘要计算：是否对 11 位纯数字手机号计算、有无空格或前缀、有无加盐、算法是否与 encryptionType 一致

任何异常排查请一并提供响应中的 head.logId，便于我方全链路定位。

6.4 联调自检清单

- [ ] 域名、appKey、appSecret、环境匹配无误，出口 IP 已报我方加白
- [ ] 待签名串严格按 ASCII 升序拼接、剔除空值、UTF-8、MD5 小写
- [ ] 已用 2.3「完整示例」复算 sign，与文档给出的值一致
- [ ] encryptionType 与 mobile 形态一致；传摘要时，17325421010 的摘要与 3.1.1 示例一致
- [ ] 已知悉本接口不需要姓名与身份证号
- [ ] 能正确解析 head.errorCode 与 body.code 两级状态
- [ ] 已知悉 999 查无结果同样计费，业务侧计费预估按有效查询次数测算
- [ ] 已实现对 result.range 字符串的二次 JSON.parse 解析、score 字符串转整数与字段容错
- [ ] 已实现超时重试（依赖幂等，不重复计费）

附录：术语表

术语	说明
appKey	公开账户标识，随请求明文传输
appSecret	加签密钥，仅本地保存用于计算 sign，切勿泄露或随请求传输
sign	对 body 业务参数按 2.3 算法计算的 32 位小写 MD5，置于请求信封
encryptionType	手机号传入形态：1 明文（默认）/ 2 MD5 摘要 / 3 SHA256 摘要
logId	全链路追踪 ID（= head.logId），排障/对账唯一凭据
range	申请反欺诈评分结果的 JSON 字符串，需二次解析（结构见 3.1.5）
score	申请反欺诈评分，200~800，分值越大风险越高
有效查询	产出了确定结论的查询（body.code = 001 或 999），即本服务的计费单位